

PRA-vLLM: Separating Query-Addressed Logical Memory from Paged KV Placement

Jorge Simão
EInnovator R&D

August 2026

Abstract

vLLM virtualizes where sequential key-value (KV) blocks reside. Progressive Retrieval Attention (PRA) virtualizes a different choice: which subset of a logically large, non-prefix memory is active for a query. We define a capability-honest integration boundary between these mappings, implement a stable logical PRA block namespace and residency state machine, and add a vLLM V1 adapter that remains an E0 facade unless an installed executor actually consumes selected native K/V. On an Apple M5 with vLLM 0.28.0 and the vLLM-Metal plugin, a 25-request smoke study separates exact-prefix reuse from selected-memory recovery. A selected 365-token prompt and the 1,577-token full context both recover the codeword in 5/5 requests; the selected condition uses 76.9% fewer visible tokens and reduces cold time to first token (TTFT) from 776.6 to 80.7 ms. Warm TTFT is 46.2 versus 52.9 ms, while vLLM reports an aggregate prefix-cache hit rate of 75.3–83.4%. These measurements establish a strong black-box baseline, not native PRA-vLLM. The installed Metal path does not yet consume non-prefix selected K/V through a verified connector or attention interface. The paper therefore closes the environment and prefix-coexistence gates while leaving the engine-native value claim open.

1 Introduction

Long-running LLM services face two related but distinct memory problems. Sequential request KV must be allocated, paged, reused, and evicted. Separately, a retained corpus or session history need not participate in every query. The first problem is physical. The second is logical.

PagedAttention and vLLM address the physical problem by decoupling logical sequence positions from non-contiguous cache blocks and scheduling requests over the resulting pool [1]. PRA addresses the logical problem by separating address state from detail state and activating only selected non-prefix memory. The combined mapping is

logical PRA memory $\rightarrow$ query-selected logical blocks $\rightarrow$ physical engine blocks.

The middle arrow cannot be inferred from ordinary prefix equality. Two requests may share the same resource but select different intervals; two physical cache slots may hold the same immutable detail while using request-local position metadata.

This paper asks whether exposing that middle mapping to vLLM improves the quality-adjusted memory, latency, and throughput frontier beyond an optimized route-then-materialize baseline. The current iteration makes three contributions:

1. a stable block identity and validated residency lifecycle independent of vLLM’s physical block numbers;
2. a vLLM adapter whose advertised PRA level is derived from an installed native executor, with tenant-scoped prefix-cache salt support at E0;
3. a pinned Apple-Silicon smoke study that quantifies selected-text and full-context behavior while explicitly recording the unmeasured native path.

2 Two Independent Reuse Systems

Automatic Prefix Caching (APC) reuses an exact token prefix. PRA selects a non-prefix logical resource or interval. These mechanisms can coexist:

$$\underbrace{\text{stable conversation prefix}}_{\text{APC}} + \underbrace{\text{query-selected retained memory}}_{\text{PRA}}.$$

The security scopes are also independent. Prefix cache presence does not grant resource authorization, and a PRA block selected for one tenant cannot be reused by another merely because its physical bytes remain resident.

Our E0/G10 baseline performs selection before the engine, injects the selected text into the current turn, and lets ordinary vLLM execute the request. This is the baseline a deeper integration must beat. Paged KV being active inside vLLM does not by itself make this path native PRA.

3 Logical PRA Block Contract

The implementation in `src/prahf/engine_memory.py` defines identity with the following fields:

Field group	Meaning
scope	tenant, optional session, and authorization scope
resource	resource identifier, immutable version, and record type
interval	half-open source-token interval $[i, j)$
model	immutable model revision, consumer layer, dtype, and K/V layout
policy	materialization profile and source-position/rebinding policy

The canonical JSON representation is hashed to produce an engine-safe logical key. Physical vLLM block numbers are deliberately absent. A resource update invalidates every matching physical realization without changing the identity of unrelated versions.

The state machine is

$$\begin{aligned} \text{INDEXEDONLY} &\rightarrow \text{OFFDEVICE} \rightarrow \text{PREFETCHING} \rightarrow \text{RESIDENT}, \\ \text{RESIDENT} &\leftrightarrow \text{PINNED}, \quad \text{RESIDENT} \rightarrow \text{EVICTABLE} \rightarrow \text{OFFDEVICE}, \end{aligned}$$

with an explicit terminal INVALID state. A transition to resident or pinned is rejected unless the engine bridge supplies at least one physical handle. Selection checks tenant and authorization scope before incrementing reuse counters. Snapshots report address bytes, logical detail bytes, resident detail bytes, indexed-only detail bytes, selections, and reuses separately.

4 Capability-Honest vLLM Adapter

The adapter in `src/pravllm/adapter.py` has two forms. Without a native executor it is an E0 OpenAI-compatible facade. It advertises APC and text fallback, but *not* logical references or native K/V. A deployment secret and tenant identifier derive a SHA-256 `cache_salt`; neither value is placed directly in the request.

With a native executor, the adapter becomes E2 and delegates generation, streaming, session close, and block-store access to that executor. This dependency-injection boundary is intentional. Metadata registration alone cannot promote the capability matrix. An executor must map selected identities to physical blocks and preserve K/V layout, GQA/MQA, masks, positions, consumer layers, request lifetime, and one correct attention normalization.

5 Environment and Method

The remote host was an Apple M5 MacBook Pro with 16 GB unified memory, macOS 26.4.1, and Metal 4. The environment used Python 3.12.7, vLLM 0.28.0+cpu, and vLLM-Metal 0.3.0.dev20260828085745 at revision 14705ad974863f68d00315655514f200366441bf. Although the package version contains +cpu, startup logs show the Metal plugin, MLX model loader, and native M5 paged-attention kernels. We therefore report both package and backend provenance.

Table 1: vLLM-Metal smoke results. TTFT is milliseconds. “Warm cached” is not available per request in the SSE usage payload.

Condition	Exact	Prompt	Cold TTFT	Warm TTFT	Warm cached
None	0%	24	875.9	42.2	n/a
Prefix	0%	333	116.6	49.4	n/a
Selected	100%	56	62.7	42.9	n/a
Prefix + selected	100%	365	80.7	46.2	n/a
Full context	100%	1577	776.6	52.9	n/a

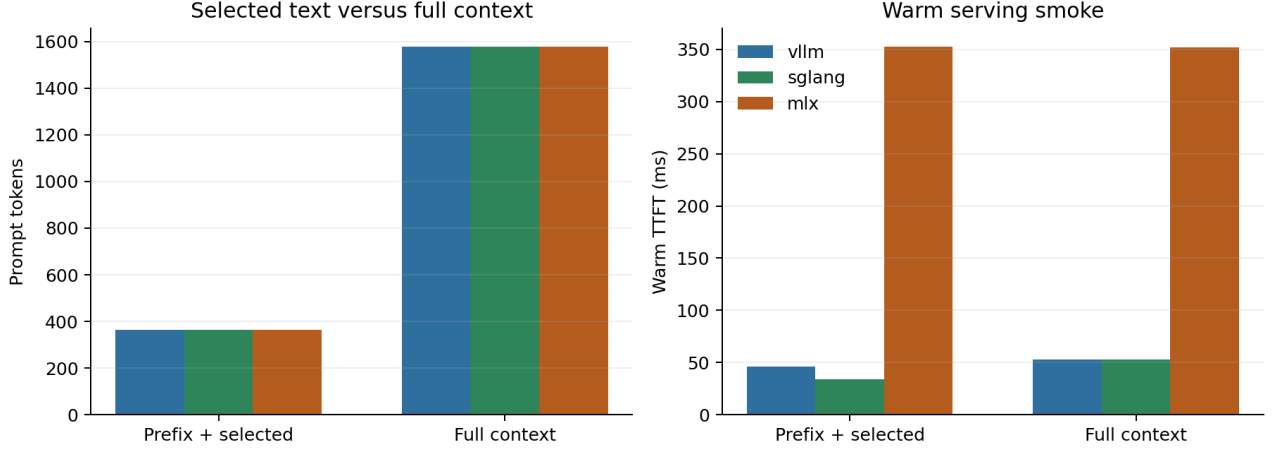


Figure 1: Shared prompt-size and warm-TTFT calibration. Timings across engines are not direct speed rankings: SGLang used an unquantized MLX conversion, whereas vLLM and MLX-LM used the 4-bit checkpoint.

The model was `mlx-community/Qwen3-0.6B-4bit` at revision `73e3e38d981303bc594367cd910ea6eb48349da8`. The server used a 2,048-token maximum context, one sequence, 20% memory reservation, and explicit APC. It allocated 1,589.1 MB for 866 KV blocks of 16 tokens, or 13,856 tokens.

The fixed synthetic task contains one authoritative codeword and twelve distractor records. Five conditions separate no memory, a stable prefix, selected evidence, stable prefix plus selected evidence, and full context. Each condition is repeated five times contiguously. The first request is reported as cold within the running server; the remaining four form the warm mean. Streaming Server-Sent Events determine TTFT. Twenty-five requests are enough for compatibility and mechanism smoke evidence, not tail percentiles or serving claims.

6 Results

Table 1 first validates the control. Neither no-memory condition recovers the codeword, whereas selected and full context recover it in every request. Prefix plus selected memory uses 365 prompt tokens versus 1,577 for full context, a reduction of

$$1 - \frac{365}{1577} = 76.9\%.$$

Cold TTFT is 80.7 ms for prefix plus selected memory and 776.6 ms for full context. This large difference includes first-touch effects and should not be treated as a steady-state speedup. Warm TTFT narrows to 46.2 versus 52.9 ms. The engine log reports aggregate APC hit rates of 75.3% and 83.4% during the run, demonstrating that selected context did not disable prefix caching.

Figure 1 shows the stable prompt-size reduction across all three tokenizers and the engine-specific latency scale. The comparable result is within-engine selected versus full context, not the absolute height of different-engine bars.

7 Promotion Gates

Gate	Status	Evidence
environment	closed	vLLM V1 with native Metal paged attention runs reproducibly
black-box semantics	smoke closed	selected and full context agree on the fixed task
prefix coexistence	smoke closed	APC remains active; repeated prompts are fast
native materialization	open	no verified non-prefix selected-K/V consumer
engine-native value	open	residency, prefetch, sharing, and block attention unmeasured
robustness	open	one model, one synthetic workload, concurrency one

The installed vLLM tree includes V1 KV connector and offload abstractions, but the observed Metal backend has not been shown to accept PRA-selected non-prefix KV while preserving source positions and attention semantics. LMCache, retrieval-aware eviction, asynchronous prefetch, cross-request sharing, and scheduler grouping therefore remain proposed experiments, not implemented results.

8 Falsification and Limitations

The central thesis fails if an optimized E0/G10 integration matches an engine-aware implementation at equal quality across memory pressure, concurrency, and reuse. The present data make that falsifier stronger: the black-box baseline already preserves quality, uses 76.9% fewer visible prompt tokens than full context, and coexists with APC. A native implementation must improve resident memory, transfer, TTFT, or throughput beyond this baseline.

The current study uses a single small 4-bit model, one Apple host, one synthetic answer, and five repeats. The cold requests are order-sensitive, per-request vLLM cached-token counts are unavailable, active PRA K/V is not measured, and no concurrency or HBM-pressure sweep was run. Apple unified memory also differs from the CUDA/LMCache environment for which many connector mechanisms were designed.

9 Related Work

vLLM introduced PagedAttention and continuous batching to reduce KV fragmentation and improve serving throughput [1]. Prefix caching reuses exact token histories; KV connectors and offload systems extend physical residency. Retrieval-augmented generation and sparse attention reduce the logical context presented to a model, but selected text is not equivalent to native non-prefix K/V. PRA combines explicit logical identity, retrieval, and native detail activation. The contribution here is the boundary between that logical mapping and vLLM’s physical mapping.

10 Conclusion

The first Paper 6 iteration establishes a reproducible vLLM-Metal baseline and a control plane that cannot accidentally overclaim native execution. Selected text recovers the fixed answer with 76.9% fewer visible tokens than full context and preserves APC. That success raises, rather than lowers, the bar for deep integration. The next decisive experiment is a verified selected-K/V connector or attention path, followed by matched-quality residency, transfer, and concurrency sweeps. Until then, PRA-vLLM is an implemented logical contract and a strong black-box baseline, not an engine-native serving result.

References

- [1] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of SOSP*, 2023.